ESCUELA SUPERIOR de Innovación y Tecnología

Contenedor Web Multiusuario con Docker Compose

Plataforma Nacional de Servicios Web Institucionales

DISEÑO

FASE 1 - ANÁLISIS Y DISEÑO DE REQUERIMIENTOS DE HOSTING MULTIUSUARIO

EQUIPO SN-18

Integrantes:

- **Luis Alonso Marín** – *Líder de Proyecto Jr.*
- **José Reynaldo García García** – *Ingeniero DevOps Docker Jr.*
- **Máximo Mariano Ramos Vásquez** – *Administrador Cloud Jr. / Desarrollador Web Jr.*
- **Gabriela Alejandra Vidal Espinoza** – *QA/Documentadora Técnica Jr.*

Enero 09, 2026

Estancia Profesional
ESCUELA SUPERIOR DE INNOVACIÓN Y TECNOLOGÍA

RESUMEN EJECUTIVO

En la Fase 1 del Proyecto 18 se realizó el análisis y diseño de una solución de hosting web multiusuario basada en contenedores Docker, orientada a la Plataforma Nacional de Servicios Web Institucionales (PNSWI). El objetivo principal de esta fase fue identificar los requerimientos del cliente y definir una arquitectura técnica que permite alojar múltiples sitios web de forma centralizada, manteniendo aislamiento lógico entre ellos.

Durante esta fase se llevó a cabo el análisis de la problemática actual, identificando la necesidad de una infraestructura compartida, escalable y fácilmente administrable. Se definieron los requerimientos funcionales y no funcionales, así como el alcance y las limitaciones del proyecto. A partir de este análisis, se propuso una arquitectura basada en una máquina virtual en la nube que ejecuta Docker Engine, utilizando Docker Compose para la gestión de servicios.

El diseño contempla contenedores web independientes para cada sitio institucional simulado, conectados mediante una red Docker tipo bridge, con volúmenes y puertos separados para garantizar persistencia y aislamiento básico. El estado actual del proyecto corresponde a una fase de diseño validado y documentado. Como riesgo identificado se considera la correcta gestión de recursos en la máquina virtual, lo cual será abordado en fases posteriores durante la implementación.

De esta manera, la Fase 1 establece una base sólida para la implementación técnica del entorno en las fases posteriores del proyecto.

Diseño Conceptual

La arquitectura conceptual del sistema se basa en una máquina virtual en la nube que actúa como host de contenedores Docker. Dentro de este host se despliegan contenedores web independientes, cada uno representando un sitio institucional simulado.

Los contenedores se conectan mediante una red Docker tipo bridge, permitiendo la comunicación interna y el aislamiento lógico entre servicios. Esta estructura permite visualizar de forma clara la separación de sitios y la administración centralizada del entorno.

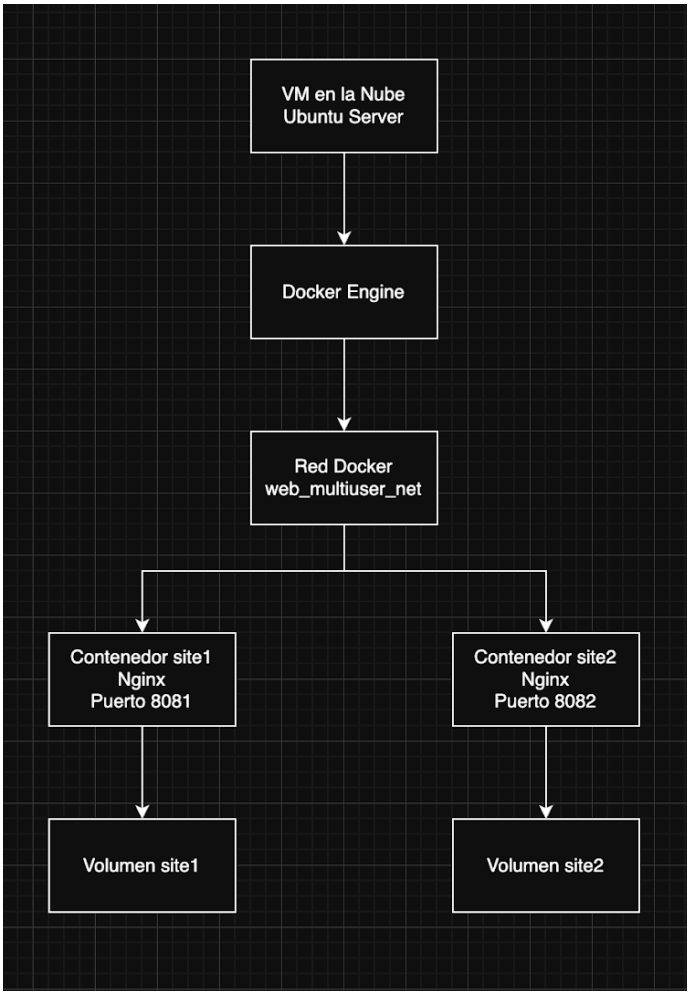


Figura 1. Arquitectura conceptual del entorno multiusuario

Fuente: Elaboración propia.

Diseño de Servicios y Trazabilidad

1. Mapa Técnico de Servicios

El sistema se compone de dos servicios web simulados, `site1` y `site2`, implementados con la imagen `nginx:latest`.

Servicio	Imagen	Mapeo de Puertos (Host:Contenedor)	Persistencia (Volumen)	Red	Propósito
site1	`nginx:latest`	8081:80	site1_data	web_net	Simulación de Sitio Institucional 1
site2	`nginx:latest`	8082:80	site2_data	web_net	Simulación de Sitio Institucional 2

2. Matriz de Trazabilidad: Requisitos y Diseño

La siguiente tabla detalla cómo las decisiones de diseño satisfacen los requisitos del proyecto:

Requisito	Decisión de Diseño	Componente Implementado
RF-01 Contenedores independientes	Contenedores independientes por sitio	Servicios Docker
RF-02 Puertos diferenciados	Asignación de puertos diferenciados	Puertos
RF-03 Orquestación centralizada	Orquestación centralizada	Docker Compose
RF-04 Persistencia de datos	Persistencia de datos	Volúmenes
RF-05 Levantar y detener servicios con un solo comando	Orquestación centralizada mediante docker-compose	docker-compose.yml

RF-06 Asignación independiente de puertos por sitio	Mapeo explícito de puertos por servicio	Configuración de puertos en docker-compose.yml
RNF-01 Uso de VM con recursos mínimos	Uso de VM con recursos mínimos	Infraestructura cloud
RNF-02 Infraestructura reproducible	Infraestructura reproducible	`docker-compose.yml`
RNF-03 Documentación técnica clara	Generación de entregables estructurados por fase (Análisis, Diseño, Implementación, Pruebas)	Documentos PDF por fase + Bitácora técnica
RNF-04 Aislamiento lógico entre sitios	Contenedores independientes con volúmenes separados por sitio	Servicios sn18_site1 y sn18_site2 en docker-compose.yml + volúmenes dedicados
RNF-05 Facilidad de mantenimiento del entorno	Uso de Docker Compose para gestión centralizada y reinicio automático (restart: unless-stopped)	docker-compose.yml + política de reinicio configurada
RNF-06 Simplicidad del entorno (sin BD ni HA)	Arquitectura mínima basada únicamente en Nginx + Docker Compose sin dependencias externas	Contenedores Nginx Alpine sobre VM Ubuntu 24.04

3. Gestión de Riesgos de Diseño y Estrategias de Mitigación

Se han identificado y mitigado los siguientes riesgos en la fase de diseño:

- **Exposición de puertos:** Se mitiga limitando el uso a los puertos mínimos indispensables.
- **Uso excesivo de recursos de la VM:** Se controla diseñando sitios sin bases de datos ni cargas intensivas, manteniendo un perfil de consumo bajo.
- **Pérdida de datos:** Se asegura la persistencia de la información mediante el uso de volúmenes de Docker.
- **Inconsistencia en la configuración:** Se aborda mediante una documentación de diseño exhaustiva antes de la implementación.
- **Exposición de información sensible (Secretos y credenciales):** Se mitiga estrictamente no almacenando secretos o credenciales en el repositorio durante esta fase inicial del proyecto.
- **Ausencia de respaldos automáticos (pérdida de configuraciones):** Se mitiga mediante la conservación del archivo docker-compose.yml y la documentación completa del entorno en el repositorio.

Arquitectura general del sistema

El sistema está compuesto por una máquina virtual en la nube que ejecuta Docker Engine como plataforma de contenedorización. La gestión del entorno se realiza mediante Docker Compose, permitiendo administrar múltiples servicios desde un solo archivo de configuración.

Servicios definidos

Se definen dos servicios principales:

- `site1`: contenedor web correspondiente al Sitio Institucional 1.
- `site2`: contenedor web correspondiente al Sitio Institucional 2.

Ambos servicios utilizan una imagen base de servidor web Nginx.

Diseño de red Docker

Los servicios se conectan a una red Docker tipo bridge, lo que permite el aislamiento lógico del entorno y la comunicación interna entre contenedores si fuera necesario.

Diseño de volúmenes

Cada sitio web cuenta con un volumen independiente que almacena su contenido HTML, garantizando la persistencia de datos y la separación entre sitios.

Mapeo de puertos

El acceso a los sitios web se realiza mediante puertos independientes:

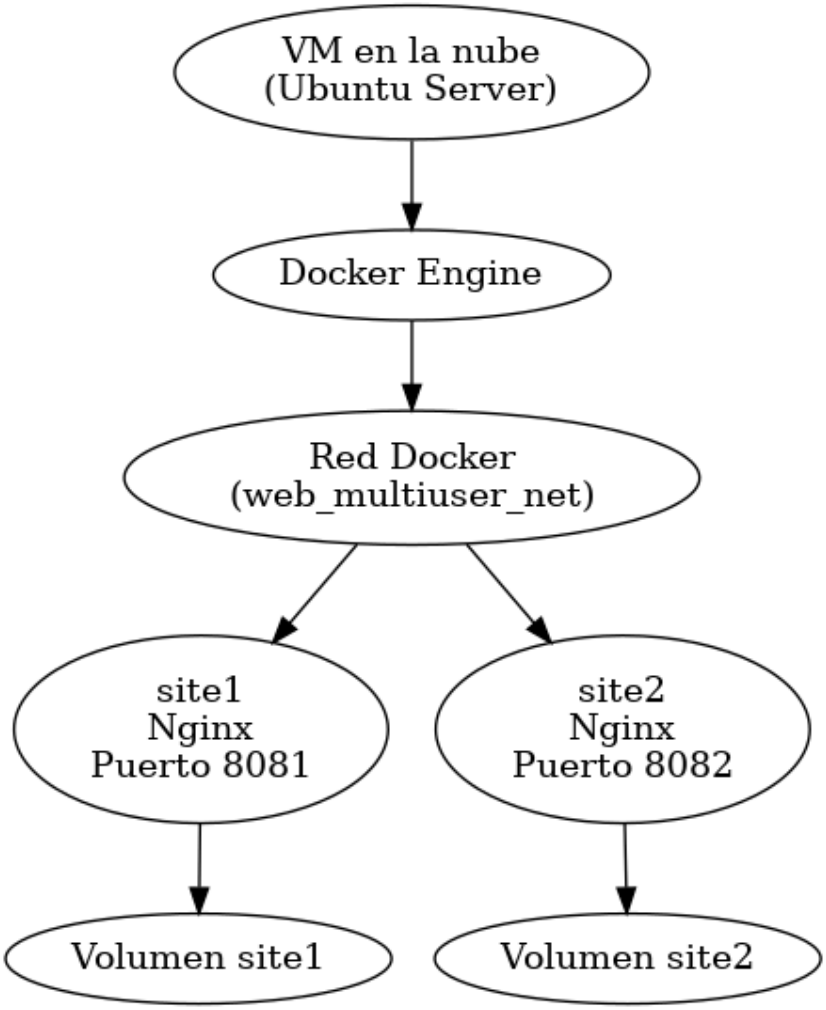
- Sitio 1: puerto 8081
- Sitio 2: puerto 8082

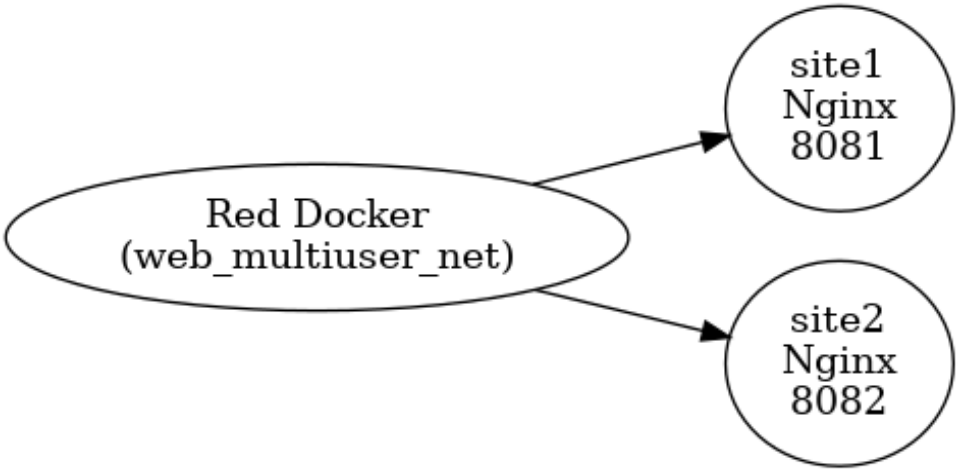
Estructura conceptual del archivo docker-compose.yml

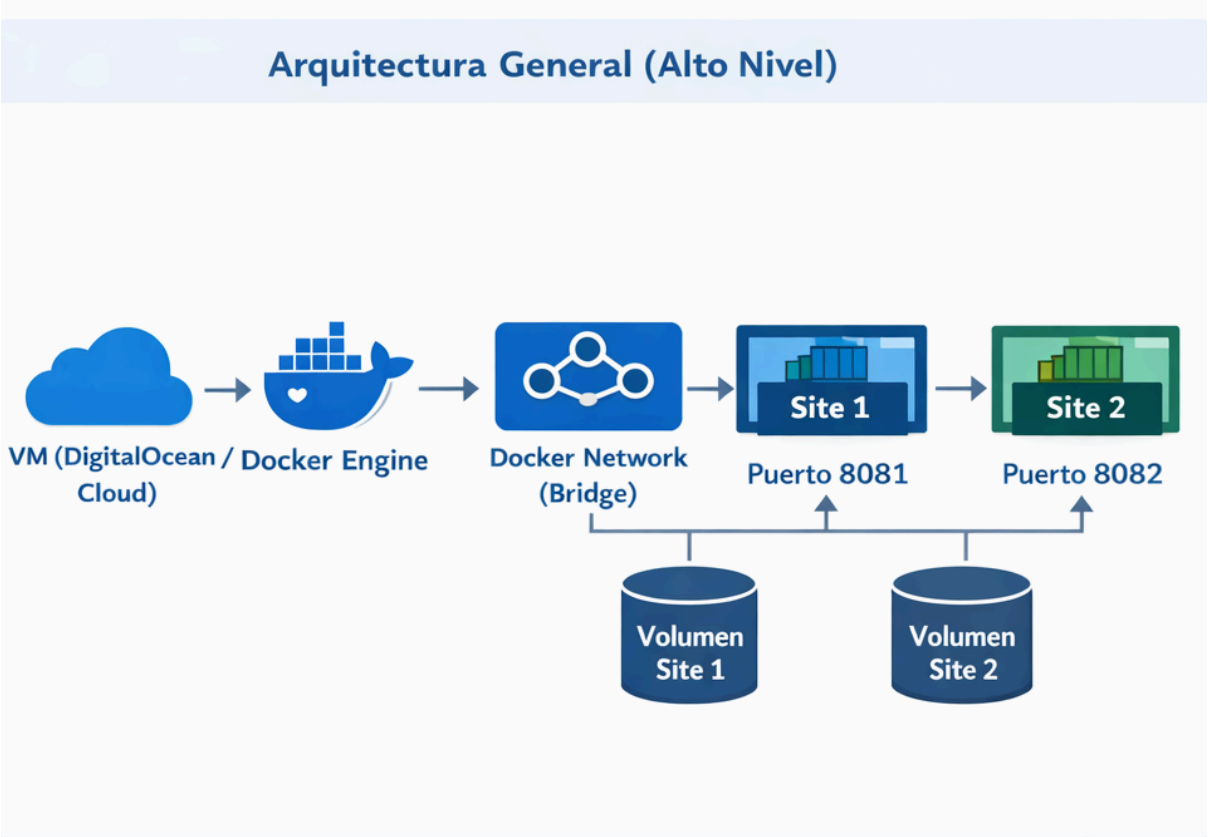
El archivo docker-compose.yml se organiza en secciones principales como services, networks y volumes, permitiendo la gestión centralizada y reproducible del entorno, facilitando su despliegue y mantenimiento.

EVIDENCIAS

- Diagrama de arquitectura conceptual del entorno multiusuario elaborado en Draw.io




site1
Nginx
8081
Red Docker
(web_multiuser_net)
site2
Nginx
8082


Arquitectura General (Alto Nivel)
VM (DigitalOcean / Cloud)
Docker Engine
Docker Network (Bridge)
Site 1
Puerto 8081
Site 2
Puerto 8082
Volumen Site 1
Volumen Site 2

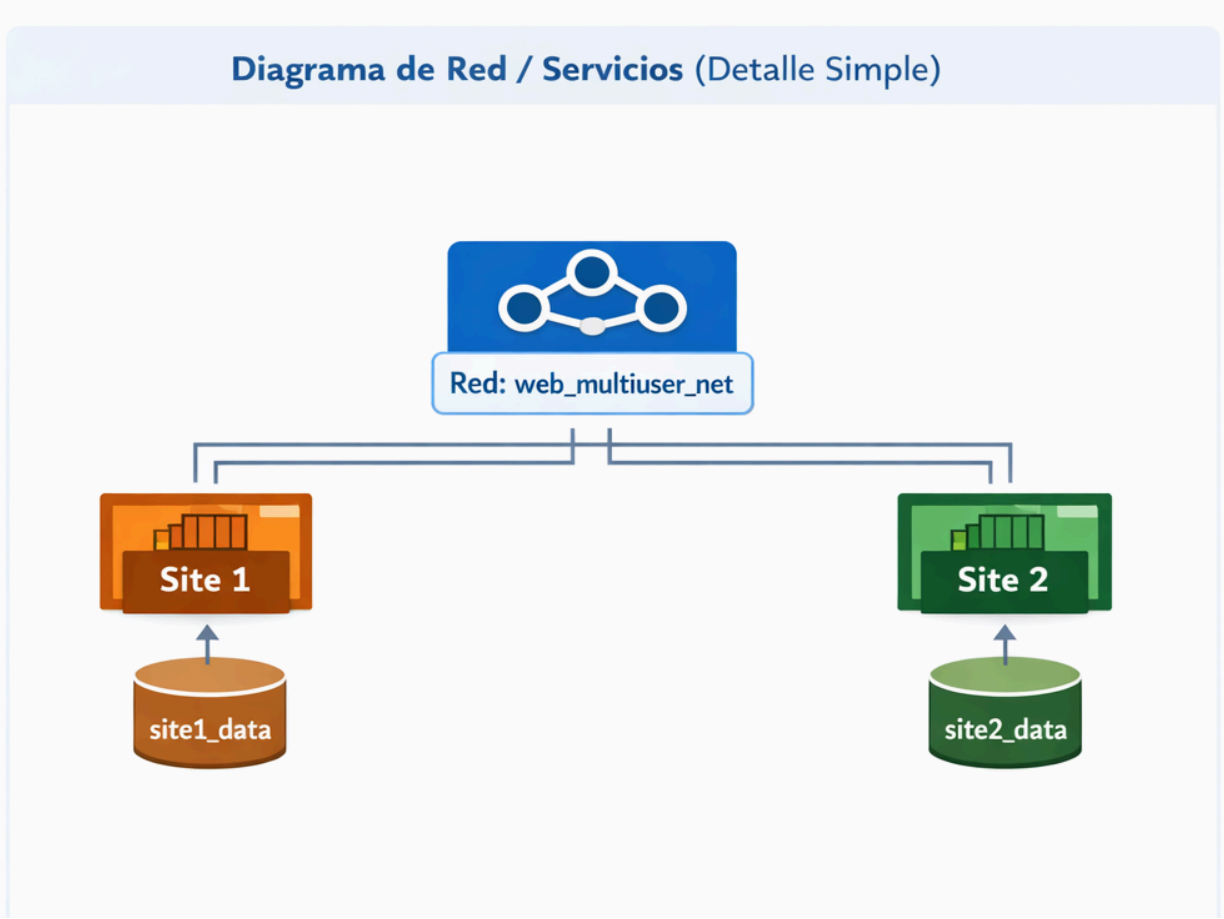
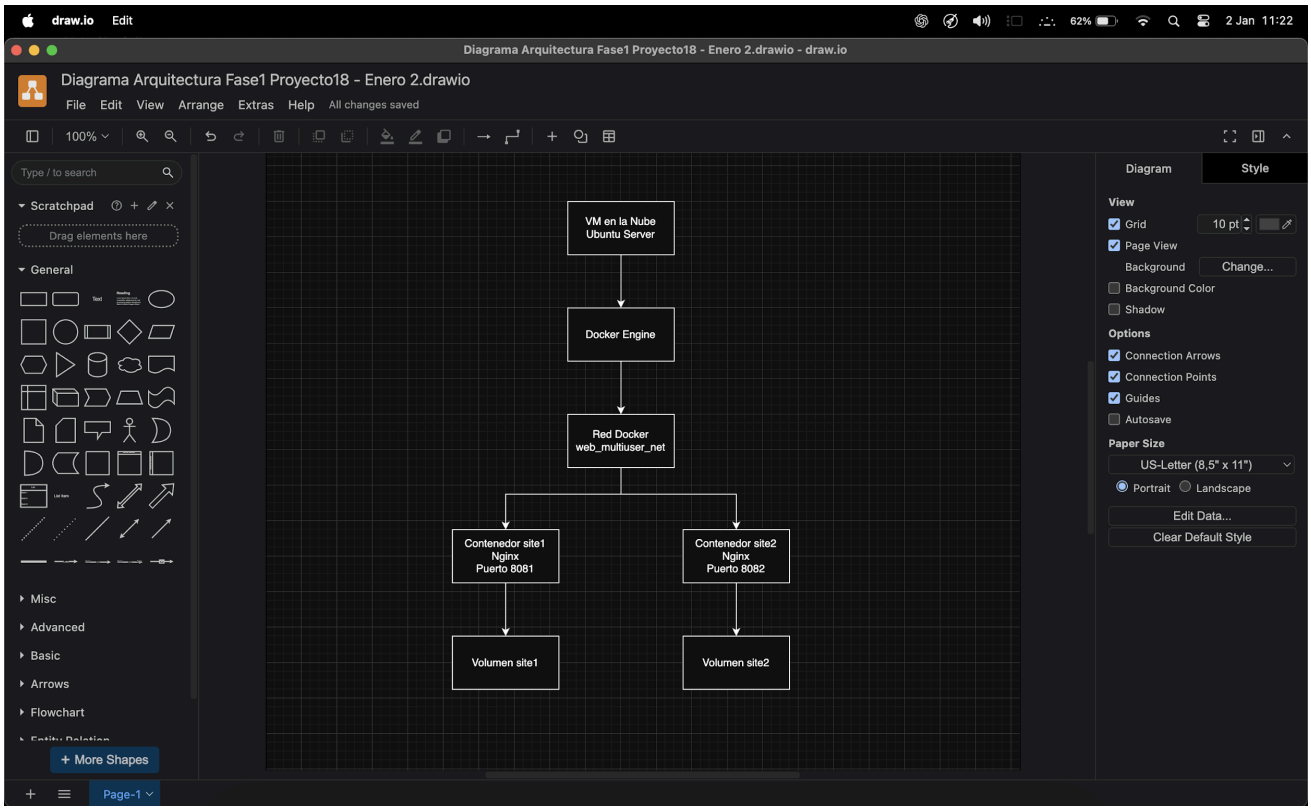


Diagrama de Red / Servicios (Detalle Simple)
Red: web_multiuser_net
Site 1
Site 2
site1_data
site2_data

- Archivo fuente del diagrama disponible en formato .drawio



- Enlace al archivo fuente del diagrama:
  - https://drive.google.com/file/d/1rnVWFDPJwn9EYT7mvxmfv5irbDTLXfoj/view?usp=drive_link

Relación con fases posteriores

El análisis de requerimientos desarrollado en la Fase 1 constituye el fundamento técnico y metodológico para el resto del proyecto. La identificación de las necesidades del cliente, la definición del escenario de hosting web multiusuario y la delimitación clara del alcance y exclusiones permiten establecer un marco de trabajo preciso para las fases subsecuentes.

Los requerimientos funcionales y no funcionales definidos en esta fase guían directamente el diseño de la arquitectura del sistema, asegurando que la solución propuesta responda a la problemática planteada y sea viable dentro de las restricciones técnicas del entorno. Asimismo, la definición de servicios, puertos, volúmenes y redes proporciona una referencia clara para la implementación práctica del entorno de contenedores.

En la Fase 2, esta información será utilizada para la construcción del archivo `docker-compose.yml`, en el cual se materializa el diseño arquitectónico definido, permitiendo el despliegue real de los sitios web en contenedores Docker. Posteriormente, las fases siguientes se apoyarán en este análisis para realizar pruebas de funcionamiento, validación del entorno y ajustes necesarios, garantizando la coherencia entre el diseño conceptual y la implementación final del sistema.

CONCLUSIÓN

Durante la Fase 1 del Proyecto 18 se logró establecer una base sólida para el desarrollo del entorno de hosting web multiusuario mediante el análisis de requerimientos y el diseño de la arquitectura del sistema. Se identificaron las necesidades del cliente, la problemática actual y las restricciones del proyecto, permitiendo definir una solución técnica adecuada al contexto planteado.

Como resultado de esta fase, se diseñó una arquitectura basada en contenedores Docker gestionados con Docker Compose, que permite el despliegue de múltiples sitios web de forma centralizada, manteniendo un aislamiento lógico entre servicios mediante el uso de redes, volúmenes y puertos independientes. Asimismo, se documentaron de manera clara los componentes del sistema y se elaboraron los diagramas correspondientes para representar la estructura conceptual y técnica del entorno.

Con la conclusión de esta fase, el proyecto se encuentra listo para avanzar a la Fase 2, en la cual se llevará a cabo la implementación del entorno diseñado, la creación del archivo docker-compose.yml y las pruebas iniciales de funcionamiento del sistema.